

Sistema de Gestión Académica (SGA-DO)

Autor: Carlos Daniel Preciado Torres

Programa Académico: Diplomado en Programación

Institución: DiplomadosOnline.com

Fecha de Entrega: 03 de agosto de 2026

INFORME DE DIAGNÓSTICO

FASE 1: ANÁLISIS DEL PROBLEMA Y PLAN DE ACCIÓN

Informe Explicativo Fase 2: Diseño de Modelado Orientado a Objetos (UML)

1. Introducción y Fundamentos del Diseño Arquitectónico

El presente documento detalla el diseño arquitectónico del Sistema de Gestión Académica (SGA-DO) mediante un diagrama de clases UML. Este modelo constituye la base estructural que guiará la implementación en los tres lenguajes de programación requeridos: Python, Java y C++. El diseño sigue los principios de la Programación Orientada a Objetos (POO), haciendo uso de herencia, polimorfismo, abstracción y composición para garantizar un sistema escalable, mantenible y alineado con los requisitos funcionales del MVP definido en la Fase 1.

2. Diagrama de Clases UML del Sistema

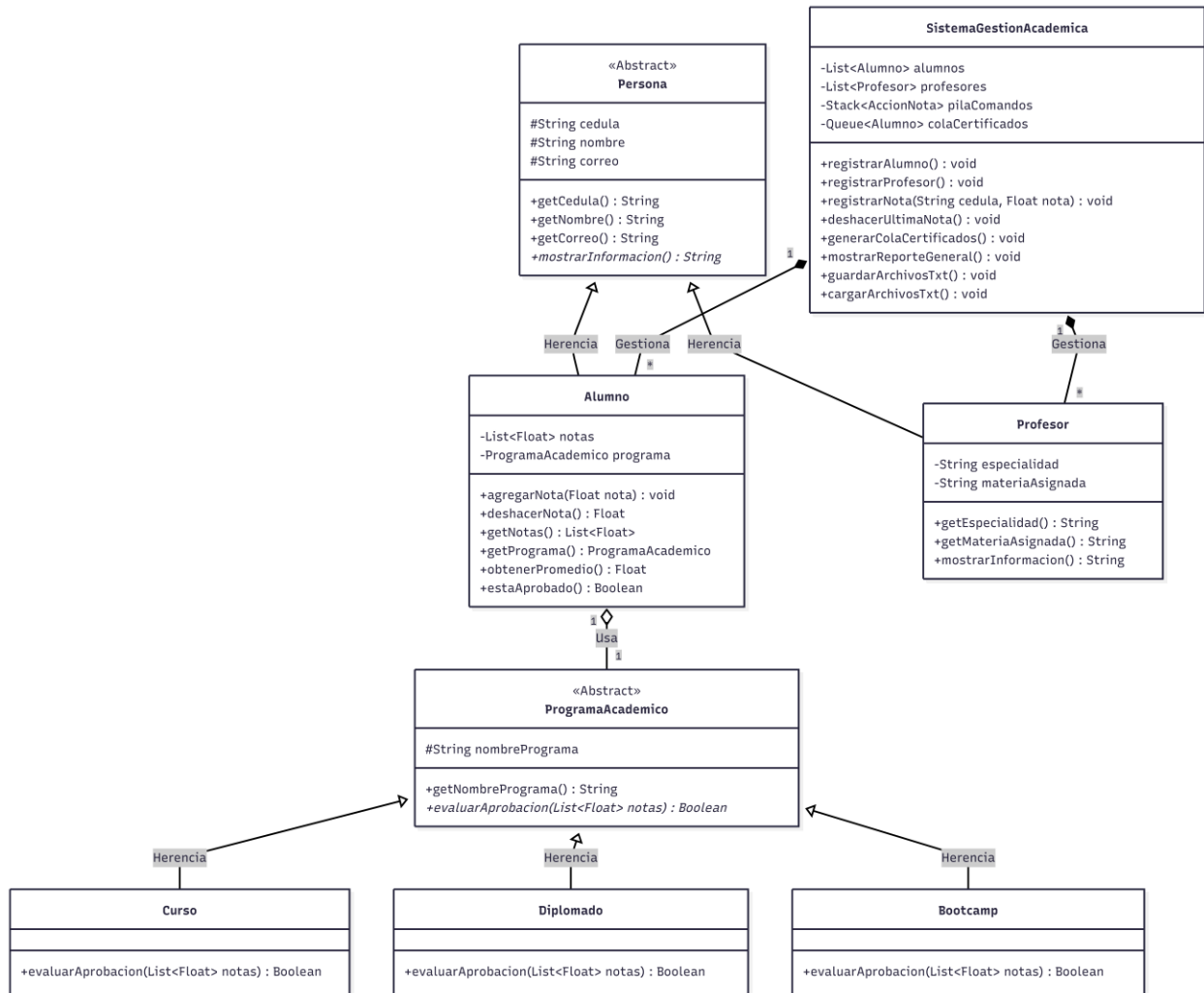
El modelo se organiza en torno a cuatro pilares fundamentales:

- Jerarquía de Personas (Persona → Alumno, Profesor)
- Jerarquía de Programas Académicos (ProgramaAcademico → Curso, Diplomado, Bootcamp)
- Clase de Gestión Central (SistemaGestionAcademica)
- Relaciones entre clases (herencia, composición, agregación y uso)

A continuación, se aprecia el diseño del modelado a objetos:

Figura 1

Diagrama de Clases UML del Sistema SGA-DO



Nota. Representación gráfica de la arquitectura orientada a objetos del SGA-DO, especificando clases abstractas, derivadas, atributos, métodos y sus relaciones estructurales.

2.3 Descripción Detallada de Clases

2.3.1 Clase Abstracta: Persona

Tabla 1

Estructura y Atributos de la Clase Persona

Atributo	Tipo	Visibilidad	Descripción
----------	------	-------------	-------------

cedula	String	# (protected)	Identificador único de la persona (ej. V-111)
nombre	String	# (protected)	Nombre completo
correo	String	# (protected)	Correo electrónico

Métodos:

- **getCedula(), getNombre(), getCorreo():** Getters estándar.
- **mostrarInformacion():** Método abstracto (*) que será sobrescrito por las subclases para mostrar detalles específicos.

Propósito: Clase base que encapsula los atributos comunes a alumnos y profesores, promoviendo la reutilización de código.

2.3.2 Clase Derivada: Alumno (hereda de Persona)

Tabla 2

Estructura y Atributos de la Clase Alumno

Atributo	Tipo	Visibilidad	Descripción
notas	List<Float>	- (private)	Lista de calificaciones (máximo 3)
programa	ProgramaAcademico	- (private)	Referencia al programa académico asignado

Métodos:

- **agregarNota(Float nota):** Añade una nota a la lista (hasta 3). Cada acción se registra en la pila de deshacer.
- **deshacerNota():** Remueve la última nota ingresada (LIFO).
- **getNotas(), getPrograma():** Getters.
- **obtenerPromedio():** Calcula el promedio de las notas.

- **estaAprobado()**: Delega la evaluación al método polimórfico del programa asociado.

Propósito: Representa al estudiante, gestiona sus calificaciones y delega la lógica de aprobación al programa académico correspondiente.

2.3.3 Clase Derivada: Profesor (hereda de Persona)

Tabla 3

Estructura y Atributos de la Clase Profesor

Atributo	Tipo	Visibilidad	Descripción
especialidad	String	- (private)	Área de especialización (ej. Python, Backend)
materiaAsignada	String	- (private)	Materia que dicta

Métodos:

- **getEspecialidad(), getMateriaAsignada()**: Getters.
- **mostrarInformacion()**: Sobrescribe el método abstracto para mostrar los datos del profesor.

Propósito: Almacena información del cuerpo docente.

2.3.4 Clase Abstracta: ProgramaAcademico

Tabla 4

Estructura y Atributos de la Clase ProgramaAcademico

Atributo	Tipo	Visibilidad	Descripción
nombrePrograma	String	# (protected)	Nombre del programa (Curso, Diplomado, Bootcamp)

Métodos:

- **getNombrePrograma()**: Getter.
- **evaluarAprobacion(List<Float> notas)**: Método abstracto (*) que implementará cada subclase con su propia regla de evaluación.

Propósito: Base para las distintas modalidades académicas, permitiendo polimorfismo en la evaluación.

2.3.5 Clases Derivadas de ProgramaAcademico

Tabla 5

Reglas de Evaluación por Modalidad Académica

Clase	Regla de Aprobación
Curso	Promedio $\geq 10/20$
Diplomado	Promedio $\geq 14/20$
Bootcamp	Ninguna nota $< 14/20$ (NO por promedio)

Método implementado: **evaluarAprobacion(List<Float> notas)** devuelve **true** si el alumno aprueba según su regla específica.

2.3.6 Clase de Gestión: SistemaGestionAcademica

Tabla 6

Estructura y Atributos de la Clase SistemaGestionAcademica

Atributo	Tipo	Visibilidad	Descripción
alumnos	List<Alumno>	- (private)	Lista de alumnos en memoria
profesores	List<Profesor>	- (private)	Lista de profesores en memoria
pilaComandos	Stack<AccionNota>	- (private)	Pila LIFO para deshacer notas

colaCertificados Queue<Alumno> - (private) Cola FIFO para graduandos

Métodos principales:

- **registrarAlumno()**: Captura datos del alumno y escribe en **alumnos.txt**.
- **registrarProfesor()**: Captura datos del profesor y escribe en **profesores.txt**.
- **registrarNota(String cedula, Float nota)**: Asigna nota a un alumno específico.
- **deshacerUltimaNota()**: Remueve la última nota usando la pila (LIFO).
- **generarColaCertificados()**: Filtra aprobados y los encola (FIFO), exportando a **certificados_pendientes.txt**.
- **mostrarReporteGeneral()**: Muestra en consola todos los alumnos y profesores.
- **guardarArchivosTxt(), cargarArchivosTxt()**: Persistencia en archivos de texto plano.

Propósito: Orquesta todas las operaciones del sistema, mantiene el estado en memoria y gestiona la persistencia.

2.4 Relaciones entre Clases

Tabla 7

Relaciones Orientadas a Objetos entre Entidades

Relación	Tipo	Descripción
Persona < -- Alumno	Herencia	Alumno es una Persona
Persona < -- Profesor	Herencia	Profesor es una Persona
ProgramaAcademico < -- Curso	Herencia	Curso es un ProgramaAcademico
ProgramaAcademico < -- Diplomado	Herencia	Diplomado es un ProgramaAcademico

ProgramaAcademico	< --	Herencia	Bootcamp es un ProgramaAcademico
Alumno "1" o-- "1" ProgramaAcademico		Composición (Agregación)	Un alumno tiene asignado un programa
SistemaGestionAcademica "1" *-- "*" Alumno		Composición	El sistema gestiona múltiples alumnos
SistemaGestionAcademica "1" *-- "*" Profesor		Composición	El sistema gestiona múltiples profesores

2.5 Justificación del Diseño

Tabla 8

Principios POO Aplicados al Modelo SGA-DO

Principio POO	Aplicación en el Modelo
Herencia	Persona → Alumno/Profesor; ProgramaAcademico → Curso/Diplomado/Bootcamp
Polimorfismo	evaluarAprobacion() se comporta distinto según la subclase (Curso, Diplomado, Bootcamp)
Abstracción	Clases abstractas Persona y ProgramaAcademico definen contratos para las subclases
Encapsulamiento	Atributos privados (-) con getters públicos para acceso controlado
Composición	SistemaGestionAcademica contiene listas de alumnos y profesores; Alumno contiene un ProgramaAcademico

2.6 Conclusión del Modelado

El diagrama de clases UML presentado constituye una guía fiel y completa para la implementación del SGA-DO. La estructura basada en herencia permite aplicar polimorfismo en la evaluación de los estudiantes, mientras que la composición facilita la gestión centralizada de los datos. El diseño garantiza:

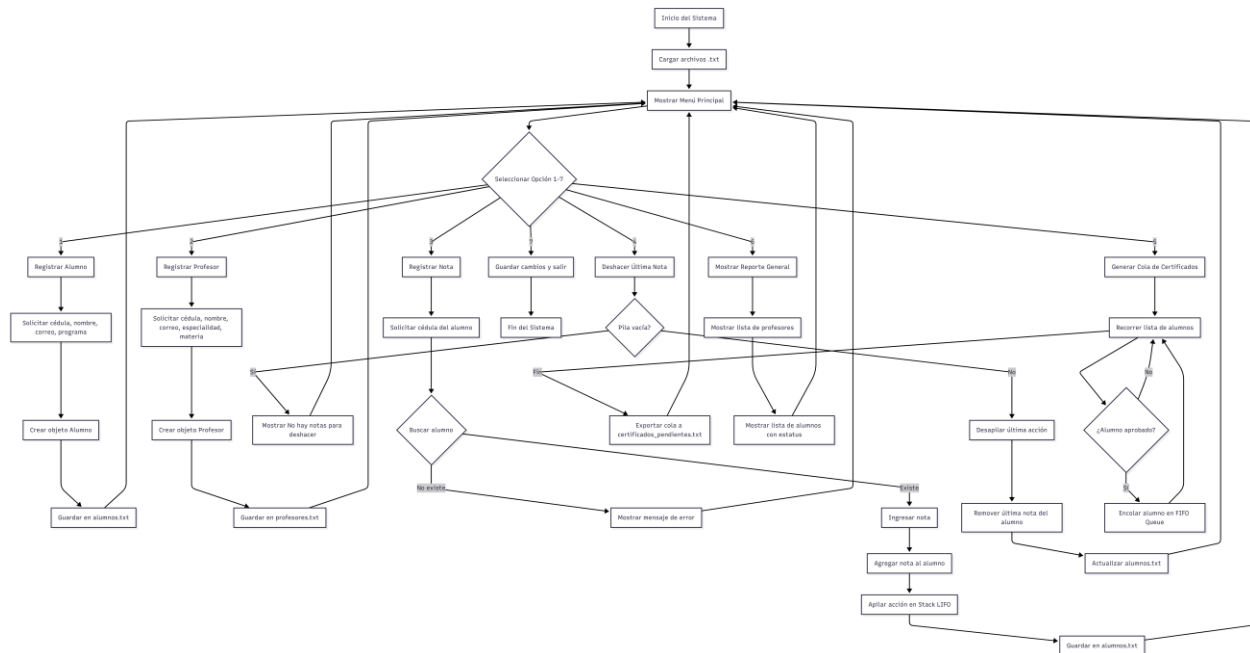
- Escalabilidad: Nuevos programas académicos pueden añadirse sin modificar la lógica central.
- Mantenibilidad: La separación de responsabilidades facilita actualizaciones y correcciones.
- Robustez: El manejo de persistencia y estructuras de datos (Pila, Cola) está claramente definido.
- Cumplimiento de requisitos: Cada requerimiento funcional del MVP tiene su correspondencia en el modelo de clases.

3. Explicación del Diagrama de Flujo

El diagrama de flujo representa el flujo de ejecución del sistema desde su inicio hasta su cierre, reflejando el comportamiento del menú interactivo definido en los requisitos funcionales:

Figura 2

Diagrama de Flujo de Procesos del Menú Interactivo SGA-DO



Nota. Muestra la navegación continua entre las 7 opciones del sistema, la interacción con la pila LIFO, la cola FIFO y la persistencia en archivos .txt.

Tabla 9

Componentes y Flujo de Operación del Sistema

Elemento	Descripción
Inicio	El sistema arranca y carga los archivos de datos (alumnos.txt , profesores.txt).
Menú Principal	Se muestra el menú de 7 opciones; el sistema permanece en bucle hasta que el usuario selecciona "Salir".
Opción 1 y 2	Registro de alumnos y profesores, con persistencia inmediata en archivos .txt .
Opción 3 y 4	Gestión de notas con funcionalidad de deshacer (Ctrl+Z) mediante estructura de pila LIFO.

Opción 5 Procesamiento de graduandos: filtra aprobados usando evaluación polimórfica, los encola (FIFO) y exporta a **certificados_pendientes.txt**.

Opción 6 Muestra en consola el reporte general de todos los registros.

Opción 7 Guarda cambios y finaliza el programa de forma segura.

El flujo demuestra cómo el sistema integra estructuras de datos (Pila, Cola), persistencia en archivos y lógica de negocio polimórfica dentro de un bucle de control continuo.

5. Conclusión Final

La Fase 2 del proyecto ha quedado completa con la entrega del modelo UML, su informe explicativo, el diagrama de flujo y el mapa mental. Este conjunto de artefactos proporciona:

- Visión clara de la arquitectura: El diagrama de clases muestra cómo se estructuran los datos y la lógica de negocio.
- Guía para la codificación: Los tres lenguajes de implementación (Python, Java, C++) seguirán este diseño común.
- Validación de requisitos: Cada requerimiento funcional del MVP está representado en el modelo.
- Documentación profesional: El informe, el flujo y el mapa mental son materiales listos para presentación ante la junta directiva de DiplomadosOnline.com.

El modelo cumple estrictamente con los lineamientos del proyecto, haciendo uso correcto de herencia, polimorfismo, encapsulamiento y composición. La implementación futura podrá desarrollarse de manera ordenada y consistente, garantizando la robustez y escalabilidad del SGA-DO.